Deployment

Цели работы:

- освоить создание и управление deployment
- изучить стратегии обновления приложений
- научиться масштабировать и откатывать развертывания
- освоить мониторинг состояния deployment

Deployment — это ресурс высокого уровня для декларативного управления приложениями в Kubernetes. Он обеспечивает развертывание, обновление и масштабирование Pod'ов.

Ключевые преимущества:

- *Автоматическое управление жизненным циклом*: Pod'ы являются эфемерными объектами. Deployment автоматически восстанавливает упавшие Pod'ы и перезапускает их при необходимости.
- *Стратегии обновления*: поддерживает различные стратегии обновления (RollingUpdate, Recreate) без простоев приложения.
- *Масштабирование*: позволяет легко увеличивать или уменьшать количество реплик приложения.
- *История и откаты*: сохраняет историю изменений и позволяет быстро откатиться к предыдущей версии.
- *Декларативное управление*: описывается желаемое состояние, а Kubernetes самостоятельно его достигает и поддерживает.

Deployment создает и управляет ReplicaSet'ами, которые в свою очередь гарантируют работу указанного количества идентичных Pod'ов

Пример 1: Создание базового Deployment

deployment-basic.txt – Блокнот

Файл Правка Формат Вид Справка

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  labels:
    app: nginx
    environment: test
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx-container
        image: nginx:1.25
        ports:
        - containerPort: 80
```

Стр 14, стлб 14 100% Windows (CRLF) UTF-8

Здесь

`selector:` Определяет, какими Pod'ами управляет этот Deployment (по метке app: nginx)

`template`: Шаблон для создания новых Pod'ов

`replicas`: Количество идентичных копий приложения

Создание и проверка Deployment

- применяем манифест
- проверяем создание Deployment
- проверяем созданные Pod'ы
- смотрим детальную информацию

```
C:\Temp>kubectl apply -f deployment-basic.yaml
deployment.apps/web-app created

C:\Temp>kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
web-app   2/2     2            2           23s

C:\Temp>kubectl get pods -l app=nginx
NAME                       READY   STATUS    RESTARTS   AGE
web-app-7d76b8c68-55q2c    1/1     Running   0          37s
web-app-7d76b8c68-g2bvx    1/1     Running   0          37s
```

```
C:\Temp>kubectl describe deployment web-app
Name:                   web-app
Namespace:              default
CreationTimestamp:      Fri, 03 Feb 2025 02:08:02 +0300
Labels:                 app=nginx
                        environment=test
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=nginx
Replicas:               2 desired | 2 updated | 2 total | 2 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx
  Containers:
   nginx-container:
    Image:        nginx:1.25
    Port:         80/TCP
    Host Port:    0/TCP
    Environment:  <none>
    Mounts:       <none>
  Volumes:        <none>
  Node-Selectors: <none>
  Tolerations:    <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
```

Тестирование приложения

- перенаправляем порт для доступа к приложению
- теперь откройте в браузере: http://localhost:8080

```
C:\Temp>kubectl port-forward deployment/web-app 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080
Handling connection for 8080
```

Ключевые моменты:

- deployment создает и управляет Pod'ами автоматически
- selector связывает Deployment с Pod'ами через метки
- template определяет конфигурацию создаваемых Pod'ов
- replicas гарантирует нужное количество работающих экземпляров
- port-forward позволяет тестировать приложение локально

Пример 2: Стратегии обновления

Создание Deployment со стратегией RollingUpdate

*deployment-rolling.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rolling-app
  labels:
    app: webserver
    version: "1.0"
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  minReadySeconds: 10
  selector:
    matchLabels:
      app: webserver
  template:
    metadata:
      labels:
        app: webserver
        version: "1.0"
    spec:
      containers:
      - name: web-container
        image: nginx:1.25
        ports:
        - containerPort: 80
```

Стр 30, стлб 8 | 90% | Windows (CRLF) | UTF-8

Здесь указаны следующие параметров стратегии:

– **`type: RollingUpdate:`** Постепенное обновление без простоя
– **`maxSurge: 1`**: Максимум на 1 Pod больше желаемого количества (может быть 4 Pod'a вместо 3 во время обновления)

– **`maxUnavailable: 0`**: Ни один Pod не должен быть недоступен во время обновления
– **`minReadySeconds: 10`**: Pod считается готовым только после 10 секунд работы

Создание и проверка Deployment

– создаем Deployment
– проверяем состояние
– запускаем наблюдение за Pod'ами в отдельном терминале

```
C:\Temp>kubectl apply -f deployment-rolling.yaml
deployment.apps/rolling-app created

C:\Temp>kubectl get deployment rolling-app
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
rolling-app   3/3     3            3           22s

C:\Temp>kubectl get pods -l app=webserver
NAME                           READY   STATUS    RESTARTS   AGE
rolling-app-6f9468b57b-5llfn   1/1     Running   0          31s
rolling-app-6f9468b57b-mlk4d   1/1     Running   0          31s
rolling-app-6f9468b57b-vxxrf   1/1     Running   0          31s
```

```
C:\Temp>kubectl get pods -l app=webserver --watch
NAME                           READY   STATUS    RESTARTS   AGE
rolling-app-6f9468b57b-5llfn   1/1     Running   0          70s
rolling-app-6f9468b57b-mlk4d   1/1     Running   0          70s
rolling-app-6f9468b57b-vxxrf   1/1     Running   0          70s
```

Запуск обновления образа
– обновляем версию nginx
– мониторим процесс обновления

```
C:\Temp>kubectl set image deployment/rolling-app web-container=nginx:1.26
deployment.apps/rolling-app image updated

C:\Temp>kubectl rollout status deployment/rolling-app
Waiting for deployment "rolling-app" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "rolling-app" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "rolling-app" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "rolling-app" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "rolling-app" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "rolling-app" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "rolling-app" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "rolling-app" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "rolling-app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "rolling-app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "rolling-app" rollout to finish: 1 old replicas are pending termination...
deployment "rolling-app" successfully rolled out
```

В терминале с --watch

```
C:\Temp>kubectl get pods -l app=webserver --watch
NAME                          READY   STATUS    RESTARTS   AGE
rolling-app-6f9468b57b-5llfn  1/1     Running   0          70s
rolling-app-6f9468b57b-mlk4d  1/1     Running   0          70s
rolling-app-6f9468b57b-vxxrf  1/1     Running   0          70s
rolling-app-5867bd6975-pd2tb  0/1     Pending   0          0s
rolling-app-5867bd6975-pd2tb  0/1     Pending   0          0s
rolling-app-5867bd6975-pd2tb  0/1     ContainerCreating   0          0s
rolling-app-5867bd6975-pd2tb  1/1     Running             0          25s
rolling-app-6f9468b57b-mlk4d  1/1     Terminating         0          3m23s
rolling-app-5867bd6975-sk6wx  0/1     Pending             0          0s
rolling-app-5867bd6975-sk6wx  0/1     Pending             0          0s
rolling-app-5867bd6975-sk6wx  0/1     ContainerCreating   0          0s
rolling-app-6f9468b57b-mlk4d  0/1     Completed           0          3m24s
rolling-app-5867bd6975-sk6wx  1/1     Running             0          2s
rolling-app-6f9468b57b-mlk4d  0/1     Completed           0          3m26s
rolling-app-6f9468b57b-mlk4d  0/1     Completed           0          3m26s
rolling-app-6f9468b57b-mlk4d  0/1     Completed           0          3m26s
rolling-app-6f9468b57b-5llfn  1/1     Terminating         0          3m35s
rolling-app-5867bd6975-ks2hv  0/1     Pending             0          0s
rolling-app-5867bd6975-ks2hv  0/1     Pending             0          0s
rolling-app-5867bd6975-ks2hv  0/1     ContainerCreating   0          0s
rolling-app-6f9468b57b-5llfn  0/1     Completed           0          3m37s
rolling-app-5867bd6975-ks2hv  1/1     Running             0          2s
rolling-app-6f9468b57b-5llfn  0/1     Completed           0          3m38s
rolling-app-6f9468b57b-5llfn  0/1     Completed           0          3m38s
rolling-app-6f9468b57b-vxxrf  1/1     Terminating         0          3m48s
rolling-app-6f9468b57b-vxxrf  0/1     Completed           0          3m49s
```

Проверка истории обновлений

- смотрим историю изменений
- детали конкретной ревизии

```
C:\Temp>kubectl rollout history deployment/rolling-app
deployment.apps/rolling-app
REVISION  CHANGE-CAUSE
1         <none>
2         <none>


C:\Temp>kubectl rollout history deployment/rolling-app --revision=2
deployment.apps/rolling-app with revision #2
Pod Template:
  Labels:       app=webserver
        pod-template-hash=5867bd6975
        version=1.0
  Containers:
   web-container:
    Image:      nginx:1.26
    Port:       80/TCP
    Host Port:  0/TCP
    Environment:        <none>
    Mounts:     <none>
  Volumes:      <none>
  Node-Selectors:       <none>
  Tolerations:  <none>
```

Ключевые моменты:

- Deployment успешно обновился с версии 1 на версию 2
- образ контейнера изменился на nginx:1.26
- CHANGE-CAUSE пустой потому что использовали команду set image

Создание Deployment со стратегией RollingUpdate

deployment-recreate.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: recreate-app
spec:
  replicas: 3
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: recreate-app
  template:
    metadata:
      labels:
        app: recreate-app
    spec:
      containers:
      - name: app-container
        image: httpd:2.4
        ports:
        - containerPort: 80
```

Здесь

`type: Recreate`: Все старые Pod'ы удаляются перед созданием новых

Используется для приложений, которые не поддерживают работу двух версий одновременно (например, с миграциями БД)

Тестирование стратегии Recreate

- создаем Deployment
- начинаем наблюдение (в другом окне)
- запускаем обновление
- наблюдаем процесс

```
C:\Temp>kubectl apply -f deployment-recreate.yaml
deployment.apps/recreate-app created

C:\Temp>kubectl set image deployment/recreate-app app-container=httpd:2.4.58
deployment.apps/recreate-app image updated
```

```
C:\Temp>kubectl get pods -l app=recreate-app --watch
NAME                              READY   STATUS              RESTARTS   AGE
recreate-app-7c8f5fc68b-87r7s     0/1     ContainerCreating   0          10s
recreate-app-7c8f5fc68b-g5jrc     0/1     ContainerCreating   0          10s
recreate-app-7c8f5fc68b-l75kw     0/1     ContainerCreating   0          10s
recreate-app-7c8f5fc68b-g5jrc     1/1     Running             0          32s
recreate-app-7c8f5fc68b-87r7s     1/1     Running             0          35s
recreate-app-7c8f5fc68b-l75kw     1/1     Running             0          35s
recreate-app-7c8f5fc68b-g5jrc     1/1     Terminating         0          88s
recreate-app-7c8f5fc68b-87r7s     1/1     Terminating         0          88s
recreate-app-7c8f5fc68b-l75kw     1/1     Terminating         0          88s
recreate-app-7c8f5fc68b-l75kw     0/1     Completed           0          91s
recreate-app-7c8f5fc68b-87r7s     0/1     Completed           0          91s
recreate-app-7c8f5fc68b-g5jrc     0/1     Completed           0          91s
recreate-app-7c8f5fc68b-l75kw     0/1     Completed           0          91s
recreate-app-7c8f5fc68b-l75kw     0/1     Completed           0          91s
recreate-app-7c8f5fc68b-87r7s     0/1     Completed           0          93s
recreate-app-7c8f5fc68b-87r7s     0/1     Completed           0          93s
recreate-app-7c8f5fc68b-g5jrc     0/1     Completed           0          93s
recreate-app-7c8f5fc68b-g5jrc     0/1     Completed           0          93s
recreate-app-78dc7c4f59-pg2c8     0/1     Pending             0          0s
recreate-app-78dc7c4f59-k4qhw     0/1     Pending             0          0s
recreate-app-78dc7c4f59-sjmd8     0/1     Pending             0          0s
recreate-app-78dc7c4f59-pg2c8     0/1     Pending             0          0s
recreate-app-78dc7c4f59-sjmd8     0/1     Pending             0          0s
recreate-app-78dc7c4f59-k4qhw     0/1     Pending             0          0s
recreate-app-78dc7c4f59-pg2c8     0/1     ContainerCreating   0          0s
recreate-app-78dc7c4f59-k4qhw     0/1     ContainerCreating   0          0s
recreate-app-78dc7c4f59-sjmd8     0/1     ContainerCreating   0          0s
recreate-app-78dc7c4f59-sjmd8     1/1     Running             0          33s
recreate-app-78dc7c4f59-k4qhw     1/1     Running             0          34s
recreate-app-78dc7c4f59-pg2c8     1/1     Running             0          36s
```

Сравнение стратегий

RollingUpdate (рекомендуется для веб-приложений)

Преимущества:	**Недостатки:**
Нет простоя приложения	Временное потребление больше ресурсов
Постепенное обновление	Две версии приложения работают одновременно
Автоматический откат при проблемах	

Recreate (для специальных случаев)

Преимущества:	**Недостатки:**
Атомарное обновление	Простой приложения во время обновления
Предсказуемое поведение	Нет постепенного развертывания
Подходит для критичных изменений	

Задания для самостоятельного решения

Задание 3.1 Базовое управление Deployment
Цель: закрепить навыки работы с командами описания и диагностики Deployment.

Задача:
Создайте Deployment с именем explore-deploy, который использует образ nginx:1.25 и имеет 3 реплики.

Выполните следующие команды и проанализируйте их вывод:

```
kubectl get deployment explore-deploy -o wide
kubectl describe deployment explore-deploy
kubectl get pods -l app=nginx
kubectl logs deployment/explore-deploy --tail=5
kubectl exec -it <pod-name> -- nginx -v
```

Вопросы для отчета:
Сколько Pod'ов управляется этим Deployment?
Какая стратегия обновления используется по умолчанию?
Какой образ используется в контейнерах?
Какая версия nginx запущена в Pod'ах?
Какой селектор использует Deployment для управления Pod'ами?

Задание 3.2 Масштабирование и обновление Deployment
Цель: освоить операции масштабирования и обновления приложений.

Задача:
На основе Deployment из задания 3.1 выполните последовательность действий:

Масштабируйте Deployment до 5 реплик: (`kubectl scale deployment explore-deploy --replicas=5`)
Обновите образ на версию 1.26:
Отследите процесс обновления:
Выполните откат на предыдущую версию:

Вопросы для отчета:
Как изменилось количество Pod'ов после масштабирования?
Сколько времени заняло обновление до версии 1.26?
Какие Pod'ы были созданы при обновлении (проверьте по именам)?
Как проверить, что откат прошел успешно?

Задание 3.3 Production-ready Deployment
Цель: научиться создавать Deployment для production-среды.

Задача:
Создайте Deployment production-app со следующими требованиями:

Требования к YAML-манифесту:
- 4 реплики приложения
- Образ: nginx:1.25
- Стратегия обновления: RollingUpdate с параметрами: maxSurge: 1, maxUnavailable: 0
- Метки: app: production, env: staging
- Ресурсы: requests: memory: "64Mi", cpu: "250m", limits: memory: "128Mi", cpu: "500m"

Проверочные команды:

```
kubectl get deployment production-app -o wide
kubectl describe deployment production-app
kubectl get replicasets -l app=production
kubectl get pods -l app=production --show-labels
```

Задание 3.4 Аварийные ситуации и восстановление
Цель: понять поведение Deployment при сбоях.

Задача:

Создайте Deployment с 3 репликами

Сымитируйте аварийные ситуации:
А) Удалите один Pod вручную и понаблюдайте за автоматическим восстановлением

Б) Создайте проблемное обновление: обновите на несуществующий образ, наблюдайте за состоянием, выполните откат:

Вопросы для отчета:
Сколько времени заняло автоматическое восстановление Pod'a?
Какое состояние у Pod'ов при проблемном обновлении?
Как Deployment обрабатывает ошибки обновления образов?
Какие события отображаются в describe при ошибках?

Задание 3.5: Мониторинг и диагностика
Цель: освоить инструменты мониторинга состояния Deployment.

Задача:
Для созданного Deployment выполните диагностику:

История изменений
Текущий статус развертывания
События Deployment
```
kubectl describe deployment production-app | grep -A 10 Events
```
Логи всех Pod'ов Deployment
Использование ресурсов
```
kubectl top pods -l app=production
```

Вопросы для отчета:
Сколько ревизий хранится в истории Deployment?
Какие команды вызывали изменения в Deployment?
Есть ли предупреждения или ошибки в событиях?
Какое потребление ресурсов у Pod'ов Deployment?